

## CODEx DEPLOYMENT BLUEPRINT

---

Complete implementation guide for ChatGPT / Codex  
24 structured prompts · 12 implementation sprints · Full code specifications  
\$1,000,000 Year 1 Target · SMB-First Strategy

---

# HOW TO USE THIS BLUEPRINT WITH CODEX / CHATGPT-5.5

## SYSTEM PROMPT TO PASTE AT THE START OF EVERY CODEX SESSION:

You are the Lead Architect implementing JARVIS vNEXT for Aliyar Solutions. CEO: Syed Abrar (Captain). You build the system described in the architecture document. Tech stack: FastAPI + SQLAlchemy 2.0 async + PostgreSQL 16 + Redis 7 + React 18 + Vite + Tailwind CSS. AWS region: ap-south-2 (Hyderabad). Branch: claude/jarvis-cans-api-integration-ZThTD. Every table has tenant\_id UUID. RLS from Day 1. Multi-tenancy built in. Never commit .env. All secrets in AWS Secrets Manager. Revenue target: \$1,000,000 Year 1. Strategy: SMB first (clients 1-20), then enterprise.

## Blueprint Structure

This blueprint is organized into 12 implementation sprints (Weeks 1–12). Each sprint contains: (1) exact Codex prompts to paste, (2) expected file outputs, (3) test commands to verify, (4) integration checkpoints. Complete each sprint before moving to the next.

| SPRINT    | WEEKS | FOCUS                                                | REVENUE GATE       |
|-----------|-------|------------------------------------------------------|--------------------|
| Sprint 1  | 1     | Database schema + tenant_id + multi-tenancy          | Foundation         |
| Sprint 2  | 1     | Lead discovery + Apollo API + scoring engine         | Leads flowing      |
| Sprint 3  | 2     | Outreach engine + 3-email sequence + personas        | First outreach     |
| Sprint 4  | 2     | Email tracking + follow-up queue + reply handler     | Full outreach loop |
| Sprint 5  | 3     | Demo builder + proposal generator + Captain approval | Close first deal   |
| Sprint 6  | 3     | Invoice engine + client portal + delivery tracker    | First invoice      |
| Sprint 7  | 4     | AI Council + agent performance + memory system       | System learns      |
| Sprint 8  | 5     | Observability: Grafana + Prometheus + Loki           | Zero blind spots   |
| Sprint 9  | 6     | ECS Fargate migration + Terraform apply              | Cloud-native       |
| Sprint 10 | 7–8   | White-label + RLS + multi-tenant API keys            | Agency revenue     |
| Sprint 11 | 9–10  | Content engine + SEO + tech radar + research         | Intelligence live  |

|           |       |                                                       |             |
|-----------|-------|-------------------------------------------------------|-------------|
| Sprint 12 | 11–12 | React frontend: all 25 views +<br>real-time dashboard | Full system |
|-----------|-------|-------------------------------------------------------|-------------|

## SPRINT 1 — Database Schema & Multi-Tenancy

GOAL: Create the unified JARVIS vNEXT database schema. Every table has tenant\_id. PostgreSQL Row-Level Security (RLS) enforces tenant isolation. SQLAlchemy 2.0 async models.

CODEx  
PROMPT #1

### Base Model + Tenant Architecture

You are implementing JARVIS vNEXT. Create the SQLAlchemy 2.0 async base model structure. Requirements: - File: backend/app/models/base.py - Base class with: id (UUID, primary key), tenant\_id (UUID, not null, indexed), created\_at (datetime), updated\_at (datetime) - Use: from sqlalchemy.orm import DeclarativeBase, mapped\_column, Mapped - Use: from sqlalchemy import UUID as SUUID, func - All models inherit from JarvisBase - tenant\_id must be on EVERY model - no exceptions Also create: backend/app/core/database.py - AsyncEngine with postgresql+asyncpg - AsyncSession with expire\_on\_commit=False - get\_db() async context manager - create\_tables() startup function Also create: backend/app/models/tenant.py - Tenant model: id, name, slug (unique), plan\_tier (enum: STARTER/GROWTH/ENTERPRISE/INDUSTRY\_OS), api\_key\_hash, settings (JSON), is\_active, created\_at - User model: id, tenant\_id, email (unique per tenant), hashed\_password, role (enum: CAPTAIN/ADMIN/VIEWER), is\_captain (bool), last\_login Tech stack: Python 3.11, SQLAlchemy 2.0, PostgreSQL 16, asyncpg driver.

CODEx  
PROMPT #2

### Core CRM + Sales Schema

Continue JARVIS vNEXT implementation. Create the CRM database models. Files to create in backend/app/models/: - lead.py: Lead model with fields: id, tenant\_id, company\_name, contact\_name, email, phone, country, industry, score (float 0-100), status (enum: NEW/CONTACTED/REPLIED/DEMO/PROPOSAL/WON/LOST), source, pain\_points (JSON array), enrichment\_data (JSON), apollo\_id, assigned\_persona, outreach\_count, last\_contact, notes - outreach.py: OutreachLog model: id, tenant\_id, lead\_id (FK), channel (enum: EMAIL/LINKEDIN/WHATSAPP), subject, body\_text, sent\_from\_persona, sent\_at, status (SENT/DELIVERED/OPENED/CLICKED/REPLIED/BOUNCED), sequence\_step (int 1-3) EmailTracking model: id, tenant\_id, outreach\_id (FK), opened\_at, clicked\_at, replied\_at, bounce\_reason FollowUpQueue: id, tenant\_id, lead\_id, sequence\_step, scheduled\_at, executed\_at, status - revenue.py: Client model: id, tenant\_id, company\_name, contact\_name, email, package\_tier, mrr\_usd (float), status (ACTIVE/PAUSED/CHURNED), started\_at Invoice model: id, tenant\_id, client\_id (FK), invoice\_number (format: ALY-YYYYMM-XXXX), amount\_usd, status (DRAFT/SENT/PAID/OVERDUE), due\_date, paid\_at, pdf\_url - approval.py: ApprovalRequest: id, tenant\_id, action\_type, title, summary, risk\_level, payload (JSON), status (PENDING/APPROVED/REJECTED), raised\_by, decided\_by, decided\_at AuditLog: id, tenant\_id, user\_id, action, entity\_type, entity\_id, before\_json, after\_json, ip\_addr All models import from app.models.base.JarvisBase. Register all in backend/app/models/\_\_init\_\_.py.

CODEx  
PROMPT #3

### Row-Level Security (RLS) Migration

Create Alembic migration that applies PostgreSQL Row-Level Security to all JARVIS tables. Requirements: - File: backend/alembic/versions/0001\_rls\_setup.py - For EVERY table: enable RLS, create policy that WHERE tenant\_id = current\_setting('app.current\_tenant\_id')::uuid - Also create function: set\_tenant\_context(tenant\_uuid) that sets app.current\_tenant\_id - Add index on tenant\_id for every table (idx\_{table}\_tenant\_id) Also modify backend/app/middleware.py: - Add TenantContextMiddleware that extracts tenant\_id from JWT token and calls set\_tenant\_context() - Add X-Request-ID middleware (generate UUID if not present, add to response headers) - Add response time header X-Response-Time Test migration: alembic upgrade head Verify RLS: INSERT with tenant\_id A, query with tenant\_id B context, should return 0 rows.

## SPRINT 2 — Lead Discovery Engine

CODEx  
PROMPT #4

### Apollo API Integration + Lead Discovery

Implement the JARVIS Lead Discovery Engine. This is the system that fills the sales pipeline. File: backend/app/services/leads/discovery.py Class: LeadDiscoveryEngine Methods: - async search\_apollo(query, filters) -> list[dict] Uses APOLLO\_API\_KEY from settings. Apollo endpoint: /people/search Filters: person\_titles=['CEO','Founder','Director','VP'], locations by country Returns: list of enriched contact records - async enrich\_company(domain) -> dict Apollo /organizations/enrich endpoint. Gets: size, revenue, tech\_stack, social\_urls - async score\_lead(lead\_data) -> float Use Gemini Pro (RESEARCH task type) with this prompt: "Score this lead 0-100 for Aliyar Solutions. ICP: SMB companies (\$500K-\$10M revenue, 5-100 employees), English-speaking, in UK/UAE/USA/AUS/Canada, with pain: manual processes, no AI systems, no tech team. Lead: {lead\_data}. Return only a number 0-100." - async run\_daily\_discovery(tenant\_id, targets: list) -> int Discovers leads for given targets (industries, countries). Scores each. Saves scored leads to DB. Returns count. - async discover\_from\_google\_maps(query, location) -> list[dict] Uses GOOGLE\_MAPS\_API\_KEY. Places API: /textsearch/json Returns businesses with name, address, phone, website Also create: backend/app/api/v1/routes/discovery.py Routes: POST /discover/run, POST /discover/leads/score, GET /discover/leads/today Register in: backend/app/api/v1/\_\_init\_\_.py

CODEx  
PROMPT #5

### Lead Scoring + ICP Matching

Implement the JARVIS Ideal Customer Profile (ICP) scoring system. File: backend/app/services/leads/scoring.py ALIYAR\_ICP = { "target\_industries": ["e-commerce", "real estate", "consulting", "professional services", "SaaS", "healthcare", "logistics", "education", "finance", "retail"], "target\_countries": ["UK", "UAE", "USA", "Australia", "Canada", "Bahrain", "Europe"], "company\_size": {"min\_employees": 5, "max\_employees": 200}, "revenue\_range": {"min\_usd": 500000, "max\_usd": 5000000}, "pain\_points": ["manual processes", "no automation", "poor lead generation", "no AI systems", "no tech team", "scaling problems"], "disqualifiers": ["competitor", "no budget", "too large enterprise", "government"], "min\_score\_for\_outreach": 60, } Class: LeadScoringEngine - async score\_against\_icp(lead: dict) -> tuple[float, dict] Returns (score, reasoning\_dict) Scoring weights: industry\_match=25pts, country\_match=20pts, size\_fit=20pts, pain\_point\_match=25pts, contact\_quality=10pts - async batch\_score(leads: list[dict], tenant\_id) -> list[Lead] Score all leads, save to DB, return sorted by score descending - async promote\_to\_pipeline(lead\_id, tenant\_id) -> Lead If score >= 60: set status=CONTACTED, assign persona (Darren Mitchell for sales), add to outreach queue Daily job: score all NEW leads from yesterday, promote top 20 to pipeline.

## SPRINT 3 — Outreach Engine

CODEX  
PROMPT #6

### 3-Email Sequence + Persona Engine

Implement the JARVIS Outreach Engine. This system sends cold outreach as human personas. File: backend/app/services/outreach/engine.py PERSONAS = { "darren\_mitchell": { "name": "Darren Mitchell", "email": "darren.mitchell@aliyarsolutions.com", "role": "Client Acquisition Specialist", "signature": "Darren Mitchell | Client Acquisition | Aliyar Solutions", "style": "Direct, results-focused, professional. Leads with value." } } 3-EMAIL SEQUENCE: Email 1 (Day 1): Subject: "Quick question about [Company]'s operations" - Lead with a specific observation about their business - One concrete problem they likely face - Soft CTA: "Would a 15-minute conversation make sense?" Email 2 (Day 4): Subject: "Results we delivered for similar [Industry] companies" - Reference: "Following up on my note earlier" - One specific result example (quantified: "reduced manual work by 70%") - CTA: "Happy to show you exactly how we did this" Email 3 (Day 8): Subject: "Last note from me, [FirstName]" - Acknowledge they're busy - Final value statement - Easy out: "If the timing isn't right, no worries at all" Class: OutreachEngine - async generate\_sequence(lead: Lead, tenant\_id) -> list[OutreachLog] Use Claude Opus (SALES task type) to write all 3 emails personalized to the specific lead. - async queue\_sequence(lead\_id, tenant\_id) -> None Add 3 emails to FollowUpQueue with correct scheduled\_at dates - async execute\_due\_outreach(tenant\_id) -> int Find all FollowUpQueue items where scheduled\_at <= now and status=PENDING. For each: generate email, check captain\_approval\_required(autonomy\_stage), send via SMTP (Gmail) or add to captain\_queue, update lead status. Returns count of messages sent. Also: backend/app/api/v1/routes/outreach.py Routes: POST /outreach/queue/{lead\_id}, POST /outreach/execute, GET /outreach/stats

CODEX  
PROMPT #7

### Gmail SMTP Integration

Implement Gmail SMTP sending for JARVIS outreach. File: backend/app/services/notifications/gmail\_sender.py Uses: GMAIL\_ADDRESS and GMAIL\_APP\_PASSWORD from settings (Phase 1 SMTP) Class: GmailSender - async send\_email(to: str, subject: str, body\_html: str, from\_name: str, from\_email: str) -> bool Uses aiosmtplib with gmail SMTP: smtp.gmail.com:587, STARTTLS Returns True on success, logs failure to audit\_log on error - async send\_outreach(outreach: OutreachLog) -> bool Sends the outreach email. Updates outreach.status to SENT. Records in email\_tracking table with tracking pixel URL if configured. - async check\_replies(tenant\_id) -> list[dict] Uses GMAIL\_ADDRESS credentials to check inbox via IMAP. Finds replies to our outreach emails. Returns list of reply dicts. (Phase 1: basic IMAP. Phase 2: Gmail OAuth API) Also: backend/app/services/notifications/slack.py - async notify\_slack(message: str) -> bool (uses SLACK\_WEBHOOK\_URL) Also: backend/app/services/notifications/telegram.py - async notify\_telegram(message: str) -> bool (uses TELEGRAM\_BOT\_TOKEN + TELEGRAM\_CHAT\_ID) Alert on: proposal\_accepted, meeting\_booked, payment\_received, system\_critical\_error

## SPRINT 4 — Email Tracking + Follow-Up + Reply Handler

CODEX  
PROMPT #8

### Reply Classification + Lead Progression

Implement the JARVIS Reply Handler — the system that processes prospect responses. File: backend/app/services/outreach/reply\_handler.py

```
REPLY_CLASSIFICATIONS = [ "INTERESTED", # "Yes, let's talk" / "Tell me more" / "Can we schedule a call?" "QUESTION", # Asking about pricing, process, timeline "NOT_NOW", # "Maybe later" / "Not the right time" "NO", # Hard no / unsubscribe "OUT_OF_OFFICE", # Auto-reply "UNKNOWN", # Cannot classify ]
```

Class: ReplyHandler

- async classify\_reply(reply\_text: str) -> tuple[str, float] Use Claude Opus. Returns (classification, confidence\_score)
- async process\_reply(lead\_id, reply\_text, tenant\_id) -> dict

1. Classify reply

2. Update lead status based on classification:

- INTERESTED → status=DEMO, alert Captain, pause follow-up sequence
- QUESTION → status=REPLIED, generate response using Claude Opus as Darren Mitchell persona
- NOT\_NOW → status=NURTURE, schedule re-contact in 30 days
- NO → status=LOST, stop all outreach, respect opt-out

3. Log to reply\_log table

4. Return action taken

- async generate\_response(reply\_text: str, lead: Lead, classification: str) -> str

Write response as Darren Mitchell persona. Natural, human, intelligent.

INTERESTED: Offer calendar link (or specific time slots)

QUESTION: Answer directly and professionally

Integrate: JARVIS checks Gmail inbox every 2 hours for new replies.

## SPRINT 5 — Demo Builder + Proposal Generator

CODEX  
PROMPT #9

### AI-Generated Proposal PDFs

Implement the JARVIS Proposal Generator — creates fully customized proposals as PDFs. File: backend/app/services/governance/proposal\_generator.py Uses ReportLab for PDF generation. Class: ProposalGenerator - async generate(lead: Lead, package\_tier: str, tenant\_id) -> str Returns path to generated PDF. Proposal structure: Page 1: Cover — Aliyar Solutions logo area, "Proposal for [Company Name]", date, confidential Page 2: Executive Summary — 3 paragraphs about their specific situation and how we solve it Page 3: Proposed Solution — 4-6 bullet points mapping their pain points to our capabilities Page 4: Package Details — table with selected package (STARTER/GROWTH/ENTERPRISE), features, price Page 5: Implementation Timeline — 4-week onboarding plan, milestones Page 6: Investment Summary — pricing table, payment terms (50% upfront + 50% on delivery), next steps Page 7: About Aliyar Solutions — company description, team, technology Use Claude Opus (SALES task type) to generate all text content personalized to the lead. Invoice numbering: ALY-YYYYMM-XXXX - async submit\_for\_approval(proposal\_id, tenant\_id) -> dict Calls request\_captain\_approval() from jarvis\_authority.py Notifies Captain via Slack + Telegram with PDF link Routes: POST /proposals/generate, POST /proposals/{id}/approve, POST /proposals/{id}/send Save PDFs to: AWS S3 bucket (or local /data/proposals/ in Phase 1)

CODEX  
PROMPT #10

### Captain Approval Queue + Dashboard

Implement the Captain Approval Queue — the most critical UX in the system. File: backend/app/services/governance/captain\_queue.py This is what Captain sees every morning. Every decision that needs approval appears here. Class: CaptainQueue - async add\_item(action\_type, title, summary, payload, risk\_level, tenant\_id) -> ApprovalRequest Creates ApprovalRequest. Notifies Captain via Slack + Telegram + WebSocket. Priority scoring: HIGH for pricing/contract/production deploy, MEDIUM for proposals, LOW for others. - async get\_pending(tenant\_id) -> list[ApprovalRequest] Returns all pending items sorted by: priority DESC, created\_at ASC - async approve(approval\_id, captain\_note, tenant\_id) -> dict Sets status=APPROVED. Executes the queued action. Logs to audit\_log. Notifies relevant agent. - async reject(approval\_id, reason, tenant\_id) -> dict Sets status=REJECTED. Notifies agent with reason. WebSocket endpoint: /ws/captain - On connect: send all pending approvals count + system health - On approval/rejection: broadcast to all Captain sessions - On new approval item: push notification to Captain Frontend route needed: /approvals view showing pending queue with approve/reject buttons.



## SPRINT 6 — Invoice Engine + Client Management

CODEX  
PROMPT #11

### Invoice Generator + Payment Tracking

Implement the JARVIS Invoice Engine. File: backend/app/services/governance/invoice\_engine.py Class: InvoiceEngine - async generate(client\_id, amount\_usd, description, due\_days=14, tenant\_id) -> Invoice Auto-number: ALY-YYYYMM-XXXX (XXXX = sequential per month per tenant) Generate PDF invoice using ReportLab (same style as proposals) PDF includes: Aliyar Solutions letterhead, client details, line items, payment instructions, bank details placeholder, due date, invoice number Save to S3/local. Store url in invoice.pdf\_url. - async send\_invoice(invoice\_id, tenant\_id) -> bool Email invoice PDF to client. Update status=SENT. Alert Captain. - async check\_overdue(tenant\_id) -> list[Invoice] Find invoices where due\_date < now and status != PAID. Send reminder email (max 3 reminders). Alert Captain on 7-day overdue. - async record\_payment(invoice\_id, amount, tenant\_id) -> Invoice Update status=PAID. Set paid\_at. Alert Captain (payment\_received event). Update client.mrr\_usd. Update revenue\_snapshots. Routes: POST /invoices/generate, POST /invoices/{id}/send, POST /invoices/{id}/pay GET /revenue/snapshot, GET /revenue/mrr-chart

## SPRINT 7 — AI Council + Memory System

CODEX  
PROMPT #12

### AI Council Implementation

Implement the JARVIS Intelligence Council — 8 AI members, weighted consensus. File: backend/app/services/ai/council.py

```
COUNCIL_MEMBERS = [ {"id": "strategist", "provider": "anthropic", "model": "claude-opus-4-8", "weight": 0.25, "specialty": "strategy"}, {"id": "engineer", "provider": "bedrock", "model": "claude-sonnet-4-6", "weight": 0.25, "specialty": "architecture"}, {"id": "analyst", "provider": "openai", "model": "gpt-4o", "weight": 0.15, "specialty": "analysis"}, {"id": "scout", "provider": "google", "model": "gemini-1.5-pro", "weight": 0.10, "specialty": "research"}, {"id": "speedster", "provider": "groq", "model": "llama-3.3-70b", "weight": 0.08, "specialty": "fast"}, {"id": "contrarian", "provider": "mistral", "model": "mistral-large", "weight": 0.07, "specialty": "critique"}, {"id": "economist", "provider": "zhipuai", "model": "glm-4", "weight": 0.05, "specialty": "economics"}, {"id": "innovator", "provider": "minimax", "model": "abab6.5s", "weight": 0.05, "specialty": "creative"}, ]
```

Class: IntelligenceCouncil - async

- convene(question: str, context: dict, council\_type: str) -> CouncilResult 1. Package question + context for each council member
- Parallel asyncio.gather() all 8 member votes (with timeout=30s per member)
- For members that timeout: use weight=0 for that session
- Require minimum 5 responses (quorum)
- Calculate weighted score:  $\sum(\text{member.weight} * \text{member.vote\_score})$
- Aggregate reasoning from all members
- Decision:  $\text{score} \geq 80$  APPROVE, 60-79 CAPTAIN\_REVIEW,  $<60$  REJECT
- Save to ai\_council\_sessions table
- Return CouncilResult(decision, score, reasoning, member\_votes) - async

adjust\_weights\_monthly(tenant\_id) -> None

For each member: compare their votes to actual outcomes (from audit\_log). +2% weight if prediction correct. -1% if wrong. Floor 3%, Cap 35%.

Routes: POST /council/convene, GET /council/sessions, GET /council/health

CODEX  
PROMPT #13

### 5-Tier Memory System

Implement the JARVIS Memory Architecture — 5 tiers of persistent intelligence. File: backend/app/services/memory/memory\_engine.py

Tier 1 — Working Memory (Redis):

- async set\_working(key: str, value: dict, ttl\_hours=24, tenant\_id) -> None
- async get\_working(key: str, tenant\_id) -> dict | None
- async clear\_working(tenant\_id) -> None

Tier 2 — Operational Memory (PostgreSQL, 90-day TTL):

- async store\_operational(category: str, content: str, source: str, tenant\_id) -> MemoryOperational
- async retrieve\_operational(category: str, limit=50, tenant\_id) -> list[MemoryOperational]
- async prune\_operational(tenant\_id) -> int (removes records older than 90 days)

Tier 3 — Strategic Memory (PostgreSQL + pgvector):

- async store\_strategic(category, content, tags: list, tenant\_id) -> MemoryStrategic
- Generates embedding using text-embedding-3-large before storing.
- async semantic\_search(query: str, limit=10, threshold=0.75, tenant\_id) -> list[MemoryStrategic]
- Embeds query, cosine similarity search in memory\_strategic table.
- async promote\_from\_operational(memory\_id, tenant\_id) -> MemoryStrategic

Tier 5 — Civilization Memory (Immutable):

- async record\_milestone(event\_type: str, content: str, tenant\_id) -> CivilizationMemory
- Appends SHA-256 hash. Never updates, never deletes.
- Daily job at 00:30: consolidate working → operational, prune expired operational.
- Weekly job: promote high-relevance operational records to strategic with embeddings.

## SPRINT 8 — Observability Stack

CODEX  
PROMPT #14

### Prometheus Metrics + Grafana Dashboards

Add comprehensive observability to JARVIS vNEXT. File: backend/app/middleware.py (extend existing) Add Prometheus metrics using prometheus-fastapi-instrumentator: - HTTP request rate by endpoint and status code - Request latency percentiles (p50, p95, p99) - Active DB connections - Redis memory usage Custom metrics to add: - jarvis\_leads\_discovered\_total (counter, labels: source, country) - jarvis\_outreach\_sent\_total (counter, labels: channel, persona) - jarvis\_ai\_cost\_usd\_total (counter, labels: provider, model, task\_type) - jarvis\_ai\_latency\_seconds (histogram, labels: provider, model) - jarvis\_council\_sessions\_total (counter, labels: decision) - jarvis\_approvals\_pending (gauge, labels: risk\_level) - jarvis\_clients\_active (gauge) - jarvis\_mrr\_usd (gauge) File: infrastructure/grafana/dashboards/jarvis\_main.json Create Grafana dashboard JSON with panels: - System Health (CPU, memory, request rate, error rate) - Revenue Metrics (MRR gauge, ARR gauge, new clients this month) - AI Operations (cost per day, calls per provider, latency) - Lead Pipeline (leads by stage funnel, outreach sent today) - Captain Queue (pending approvals count, last updated) Update docker-compose.yml to add prometheus scrape config for backend:8000/metrics.

## SPRINT 9 — ECS Fargate Migration

CODEX  
PROMPT #15

### Apply Terraform + ECS Deployment

Execute the JARVIS cloud infrastructure deployment. The Terraform is already written at `infra/terraform/`. Apply it. Steps to execute (run as Captain on EC2): Step 1 — Set Terraform variables: Create `infra/terraform/terraform.tfvars` (NEVER COMMIT — add to `.gitignore`): `aws_region = "ap-south-2" project = "jarvis" environment = "production" db_password = "" secret_key = ""` # ... all required variables Step 2 — Initialize and plan: `cd infra/terraform terraform init terraform plan -out=tfplan` Step 3 — Apply (creates all AWS resources): `terraform apply tfplan` Step 4 — Build and push Docker images: `aws ecr get-login-password --region ap-south-2 | docker login --username AWS --password-stdin docker build -t jarvis-backend backend/ docker tag jarvis-backend:latest :latest docker push :latest` Step 5 — Update ECS task definitions: Create ECS task definition JSON with: - Container: `jarvis-backend`, image: `ECR_URI`, port: `8000` - Environment: from AWS Secrets Manager (valueFrom `arn`) - Log driver: `awslogs`, group: `/jarvis/production/backend` - Memory: `1024 MB`, CPU: `512` Step 6 — Register task definition, update ECS service: `aws ecs register-task-definition --cli-input-json file://task-def.json aws ecs update-service --cluster jarvis-production-cluster --service jarvis-backend --task-definition jarvis-backend:latest` Step 7 — Verify deployment: `aws ecs describe-services --cluster jarvis-production-cluster --services jarvis-backend curl https://api.aliyarsolutions.com/health` File to create: `scripts/deploy_ecs.sh` — wrapper script for steps 4-7.

## SPRINT 10 — White-Label + Multi-Tenant

CODEX  
PROMPT #16

### Multi-Tenant API + Tenant Management

Implement the JARVIS white-label multi-tenant system. File: backend/app/services/tenancy/tenant\_manager.py Class: TenantManager - async create\_tenant(name, plan\_tier, admin\_email, admin\_password) -> tuple[Tenant, User] Create tenant record. Create admin user with is\_captain=False. Generate API key (32-byte random, store SHA-256 hash). Apply RLS context for new tenant. Return tenant + user. - async provision\_tenant(tenant\_id) -> None Seed default data: personas, agent configs, service catalog entries, governance rules. This runs after create\_tenant. - async get\_tenant\_from\_api\_key(api\_key: str) -> Tenant | None Hash the provided key, look up in tenant\_api\_keys table. Auth middleware update: - Accept JWT OR X-API-Key header - JWT: extract tenant\_id from sub claim - API Key: look up tenant from key hash - Set app.current\_tenant\_id in DB session context File: backend/app/api/v1/routes/tenancy.py Routes (Captain-only): POST /admin/tenants — create new tenant GET /admin/tenants — list all tenants PUT /admin/tenants/{id}/limits — update plan limits POST /admin/tenants/{id}/api-key — rotate API key Pricing per tenant tier (enforce in middleware): STARTER: max\_leads=500/mo, max\_agents=10, max\_ai\_calls=1000/day GROWTH: max\_leads=2000/mo, max\_agents=25, max\_ai\_calls=5000/day ENTERPRISE: max\_leads=10000/mo, max\_agents=64, max\_ai\_calls=unlimited

## SPRINT 11 — Intelligence Stack

CODEX  
PROMPT #17

### Tech Radar + Market Intelligence

Implement the JARVIS Intelligence Division services. File: backend/app/services/intelligence/tech\_radar.py Class: TechRadarEngine - async scan\_week(tenant\_id) -> list[TechRadarEntry] Use Gemini Pro (RESEARCH task type) to scan for emerging technologies. Categories: AI/ML, Cloud, Security, DevOps, Data, Frontend, Backend. Rings: ADOPT (proven), TRIAL (promising), ASSESS (watch), HOLD (avoid). Save to tech\_radar table. Return new entries. File: backend/app/services/intelligence/market\_intel.py Class: MarketIntelligenceEngine - async generate\_report(topics: list[str], tenant\_id) -> str Use Gemini Pro to generate market intelligence report. Topics: AI automation market, SMB technology adoption, competitor pricing changes. Save to market\_intelligence table. Return report content. - async monitor\_competitors(tenant\_id) -> list[dict] For each competitor in competitor\_profiles: Check their website, LinkedIn, pricing pages for changes. Alert Captain if pricing changes detected. APScheduler jobs to create: - tech\_radar\_scan: every Monday 06:00 - market\_intelligence\_report: every other Sunday 07:00 - competitor\_monitoring: every Monday 09:00 - morning\_briefing: every day 07:00 Morning briefing job: Compile: top 5 leads by score, revenue snapshot (MRR, pipeline), pending Captain approvals, system health summary, today's scheduled outreach. Send to Captain via Slack + Telegram. Log to briefing history.

CODEX  
PROMPT #18

### Morning Briefing + APScheduler

Implement the JARVIS APScheduler system with all production jobs. File: backend/app/services/scheduler/scheduler.py Use apscheduler[asyncio] with SQLAlchemy job store (persistent jobs survive restarts). Jobs to register: daily\_morning\_briefing — cron: 07:00 daily (IST = UTC+5:30, so 01:30 UTC) daily\_lead\_scoring — cron: 02:00 daily (IST) daily\_lead\_discovery — cron: 03:30 daily (IST) daily\_follow\_up\_check — cron: 10:00 daily (IST) daily\_memory Consolidate — cron: 00:30 daily (IST) daily\_optimization\_review — cron: 23:00 daily (IST) weekly\_outreach\_stats — cron: Monday 08:00 (IST) weekly\_pipeline\_health — cron: Sunday 20:00 (IST) weekly\_tech\_radar — cron: Monday 06:00 (IST) biweekly\_research\_report — cron: every other Sunday 07:00 (IST) monthly\_weight\_adjust — cron: 1st of month 00:00 (IST) Morning Briefing content: "JARVIS MORNING BRIEFING — {date} REVENUE: - Current MRR: \${mrr} - New leads today: {new\_leads} - Pipeline value: \${pipeline} ACTION REQUIRED ({pending\_approvals} items): {approval\_list} OUTREACH: - Scheduled today: {outreach\_count} messages - Replied this week: {reply\_count} - Top lead: {top\_lead\_name} ({top\_lead\_score}/100) SYSTEM: {system\_health\_summary}" Start scheduler in FastAPI lifespan (backend/app/main.py).

## SPRINT 12 — React Frontend (25 Views)

CODEx  
PROMPT #19

### React Frontend Architecture

Implement the JARVIS React frontend — glassmorphism design, all 25 views. Tech stack: React 18 + Vite + Tailwind CSS + Zustand + Axios + Recharts Design system (glassmorphism): - Background: linear-gradient(135deg, #0A1628 0%, #0D1F3C 50%, #0A1628 100%) - Cards: bg-white/5 backdrop-blur-md border border-white/10 rounded-xl - Accent colors: blue-500 (#0057FF), cyan-400 (#00C8FF), gold (#FFB700) - Text: white primary, gray-400 secondary File: frontend/src/App.jsx VIEWS map (25 views with their routes): dashboard → / → Executive Dashboard (MRR, pipeline, alerts, briefing) chat → /chat → JARVIS chat interface (full-page conversational AI) briefing → /briefing → Morning briefing history approvals → /approvals → Captain Approval Queue (cards with approve/reject) leads → /leads → Lead pipeline (table + kanban, filters, score badges) outreach → /outreach → Outreach log, sequence status, stats crm → /crm → Client management, contacts, companies proposals → /proposals → Proposal list, PDF previews, status invoices → /invoices → Invoice management, payment status agents → /agents → Agent registry, performance scores council → /council → AI Council session history + new session memory → /memory → Memory browser (working/operational/strategic) intel → /intel → Market intelligence, tech radar, competitor profiles discovery → /discovery → Lead discovery controls, run jobs, view today's leads tasks → /tasks → Task management across all projects projects → /projects → Client project tracker, milestones scheduler → /scheduler → APScheduler job dashboard, run/pause jobs notifications → /notifications → Notification center + WebSocket events gmail → /gmail → Gmail inbox monitor, outreach drafts voice → /voice → ElevenLabs TTS, voice briefings knowledge → /knowledge → Knowledge base browser + SOP library research → /research → Research reports, tech radar viewer governance → /governance → Audit log, rules, compliance catalog → /catalog → Service catalog (30 divisions) settings → /settings → Tenant settings, API keys, config File: frontend/src/store/useJarvisStore.js (Zustand) State: { user, tenant, notifications, captainQueue, systemHealth, ws }

CODEx  
PROMPT #20

### Executive Dashboard View

Build the JARVIS Executive Dashboard — the most important view in the system. File: frontend/src/components/dashboard/Dashboard.jsx Layout (4 metric cards at top + main content below): - Card 1: MRR (gold, big number, +% vs last month) - Card 2: Active Clients (cyan) - Card 3: Pipeline Value (blue) - Card 4: Pending Approvals (red if >0, green if 0) Main content grid: - Left (60%): - "Today's Pipeline" — top 5 leads by score, with quick actions - "Recent Activity" — last 10 audit log entries - Right (40%): - "Morning Briefing" — latest briefing content (collapsed/expandable) - "System Health" — green/red status for: API, DB, Redis, AI providers - "Captain Queue" — count of pending approvals with link to /approvals Bottom section: - MRR trend chart (Recharts LineChart, last 12 months) - Lead funnel chart (Recharts BarChart: NEW/CONTACTED/REPLIED/DEMO/PROPOSAL/WON) - AI Cost breakdown (Recharts PieChart: by provider) Real-time via WebSocket (/ws/captain): - Approval badge count updates without page refresh - System health indicators update every 30s - New alert banner on critical events

CODEx  
PROMPT #21

### Approvals View — Captain Decision Center

Build the Captain Approval Queue view — where Captain approves/rejects JARVIS actions. File: frontend/src/components/approvals/Approvals.jsx Layout: - Header: "Captain's Queue" + count badge - Filters: All / High Risk / Proposals / Infrastructure / Financial - Approval cards (sorted by priority, then age): Each card shows: - Risk badge (HIGH=red, MEDIUM=orange, LOW=blue) - Action type (e.g., "proposal\_send", "pricing\_decision") - Title and summary (first 200 chars, expandable) - Time in queue - Payload preview (JSON collapse/expand) - Two buttons: ✓ APPROVE (green) and ✗ REJECT (red) - Optional note field before confirming On approve: POST /approvals/{id}/approve → card disappears with animation On reject: POST /approvals/{id}/reject with reason → card disappears Keyboard shortcuts: A to approve focused card, R to reject, Tab to navigate Stats bar at top: Approved today, Rejected today, Average time in queue

## FINAL PROMPTS — Integration + First Client

CODEx  
PROMPT #22

### End-to-End Integration Test

Test the complete JARVIS vNEXT pipeline end-to-end before going live. Create file: scripts/integration\_test.py Test flow: 1. Create test tenant: test\_tenant = await TenantManager.create\_tenant("Test Agency", "GROWTH", ...) 2. Run lead discovery: leads = await LeadDiscoveryEngine.run\_daily\_discovery(tenant\_id, ["SaaS", "UK"]) 3. Assert: len(leads) > 0, all leads have score > 0 4. Score a lead: lead = leads[0]; scored = await LeadScoringEngine.score\_against\_icp(lead) 5. Queue outreach: await OutreachEngine.queue\_sequence(lead.id, tenant\_id) 6. Assert: FollowUpQueue has 3 records for this lead 7. Execute outreach (test mode — don't actually send): await OutreachEngine.execute\_due\_outreach(tenant\_id, dry\_run=True) 8. Simulate reply: await ReplyHandler.process\_reply(lead.id, "Yes, this sounds interesting!", tenant\_id) 9. Assert: lead.status == "DEMO" 10. Generate proposal: pdf\_path = await ProposalGenerator.generate(lead, "GROWTH", tenant\_id) 11. Assert: os.path.exists(pdf\_path) 12. Submit to Captain queue: result = await CaptainQueue.add\_item("proposal\_send", ..., tenant\_id) 13. Assert: result.status == "PENDING" 14. Run morning briefing: briefing = await generate\_morning\_briefing(tenant\_id) 15. Assert: "leads" in briefing and "revenue" in briefing All assertions with clear pass/fail output. If all pass: print "JARVIS vNEXT INTEGRATION: ALL SYSTEMS GO"

CODEx  
PROMPT #23

### Pre-Launch Checklist Script

Create the JARVIS pre-launch validation script. File: scripts/pre\_launch\_check.py Check every system required for first client: CHECKS = [ ("Database connection", test\_db\_connection), ("Redis connection", test\_redis\_connection), ("AWS Bedrock reachable", test\_bedrock), ("Anthropic API key valid", test\_anthropic), ("Apollo API key valid", test\_apollo), ("Gmail SMTP working", test\_gmail), ("Slack webhook working", test\_slack), ("Telegram bot working", test\_telegram), ("Lead discovery returning results", test\_discovery), ("Email generation working", test\_email\_gen), ("Proposal PDF generation working", test\_proposal\_gen), ("Captain approval queue working", test\_approval\_queue), ("WebSocket endpoint working", test\_websocket), ("Grafana accessible", test\_grafana), ("Prometheus scraping", test\_prometheus), ("S3 upload working", test\_s3), ("Domain resolving correctly", test\_domain), ] For each check: green ✓ or red ✗ with error message. Print summary: "X/17 systems ready" If all pass: print "JARVIS IS READY. CAPTAIN, BEGIN THE MISSION."

CODEx  
PROMPT #24

### Captain's First Client Activation Protocol

This is the final prompt — the system is built, Captain is ready to activate. ACTIVATION STEPS: 1. Run integration test: python scripts/integration\_test.py — all must pass 2. Run pre-launch check: python scripts/pre\_launch\_check.py — all must pass 3. Configure Captain's account: POST /admin/tenants with: name="Aliyar Solutions", plan\_tier="ENTERPRISE", captain\_email="syedabrarbhd@gmail.com" 4. Set autonomy stage to 1 (Captain approves every outreach): PUT /settings with: autonomy\_stage=1, max\_daily\_outreach=20, outreach\_countries=["UK","UAE","USA","AUS","Canada"] 5. Run first lead discovery manually: POST /discover/run with: industries=["SaaS","e-commerce","consulting"], countries=["UK","UAE"], limit=50 6. Review leads in dashboard /leads — set minimum score filter to 65 7. Select top 3 leads manually → click "Queue Outreach" 8. Check Captain Queue at /approvals — 3 outreach emails await approval 9. Review each email. If good → APPROVE. If needs editing → REJECT with note (JARVIS will rewrite) 10. After first 3 are approved and sent — monitor /outreach for opens and replies 11. First reply received → JARVIS will alert Captain via Slack + Telegram Go to /leads → find the lead → review JARVIS's drafted response → approve 12. First discovery call booked → JARVIS will build a demo → add to Captain Queue for approval 13. First proposal sent → JARVIS will track opens → follow up → close. CAPTAIN: THE SYSTEM IS NOW LIVE. \$1,000,000 STARTS HERE.



JARVIS vNEXT CODEX DEPLOYMENT BLUEPRINT Aliyar Solutions · CEO: Captain Syed Abrar Generated:  
June 2026 Revenue Target: \$1,000,000 Year 1 · SMB-First Strategy Classification: CAPTAIN EYES ONLY